

1. Création d'Indexes pour Optimisation

- **CREATE INDEX idx_membre_email ON MEMBRE(email_membre);** : Crée un index sur la colonne email_membre de la table MEMBRE. Cela permet d'optimiser les requêtes qui recherchent ou filtrent par email.
- **CREATE INDEX idx_paiement_date ON PAIEMENT(date_paiement);** : Crée un index sur la colonne date_paiement de la table PAIEMENT. Cela améliore la performance des recherches ou des tris basés sur la date des paiements.
- **CREATE INDEX idx_abonnement_statut ON ABONNEMENT(statut);** : Crée un index sur la colonne statut de la table ABONNEMENT. Cela permet d'optimiser les requêtes filtrant par statut (par exemple, abonnements actifs ou inactifs).

2. Vues pour Faciliter l'Accès aux Données

- **CREATE VIEW Vue_Abonnements_Actifs AS ...** : Crée une vue qui permet d'afficher uniquement les abonnements actifs, avec des détails sur l'abonnement et le membre associé (nom, prénom).
- **CREATE VIEW Vue_Paiements_Valides AS ...** : Crée une vue qui affiche les paiements validés, incluant le montant, la date, la méthode de paiement et les détails du membre. Cette vue filtre les paiements ayant le statut "validé".

3. Triggers pour Automatiser les Mises à Jour

- **CREATE TRIGGER tr_abonnement_expire BEFORE UPDATE ON ABONNEMENT ...** : Ce trigger est activé avant une mise à jour sur la table ABONNEMENT. Il vérifie si la date de fin de l'abonnement est passée (inférieure à la date actuelle), et si c'est le cas, il met à jour le statut de l'abonnement à "inactif".
- **CREATE TABLE HISTORIQUE_PAIEMENT ...** : Crée une table pour stocker l'historique des paiements, incluant l'ID du paiement, montant, date, statut et la date de modification.
- **CREATE TRIGGER tr_paiement_historique AFTER UPDATE ON PAIEMENT ...** : Ce trigger est exécuté après chaque mise à jour d'un paiement dans la table PAIEMENT. Il insère les anciennes informations du paiement (avant modification) dans la table HISTORIQUE_PAIEMENT pour garder une trace des changements.

4. Procédures Stockées

- **CREATE PROCEDURE Ajouter_Abonnement ...** : Crée une procédure stockée pour ajouter un nouvel abonnement. Elle prend en paramètre l'ID du membre, la date de début, la date de fin et le type d'abonnement, puis insère ces informations dans la table ABONNEMENT avec un statut "actif".
- **CREATE PROCEDURE Effectuer_Paiement ...** : Crée une procédure pour effectuer un paiement sécurisé. Elle prend en paramètre l'ID de l'abonnement, le montant, la méthode de paiement et la référence de transaction. Elle commence une transaction, insère le paiement dans la table PAIEMENT avec un statut "en_attente", puis engage la transaction. En cas d'erreur, elle annule la transaction et génère un message d'erreur.

5. Gestion des Utilisateurs et Rôles

- **CREATE ROLE admin;** : Crée un rôle admin avec tous les privilèges sur la base de données GestionAbonnements.
- **CREATE ROLE utilisateur;** : Crée un rôle utilisateur avec des privilèges limités : SELECT, INSERT et UPDATE sur la table MEMBRE, et SELECT, INSERT sur la table ABONNEMENT.
- **CREATE USER 'admin1'@'localhost' IDENTIFIED BY 'password';** : Crée un utilisateur admin1 avec un mot de passe spécifié et lui attribue le rôle admin.
- **CREATE USER 'user1'@'localhost' IDENTIFIED BY 'password';** : Crée un utilisateur user1 avec un mot de passe spécifié et lui attribue le rôle utilisateur.